

By: Elaf A. Saeed

Iraqi Speech Dataset Processing Pipeline

Complete Technical Report for Stages 1-10

Including Detailed Step-by-Step Workflow and Future Development Roadmap

Project	Iraqi Speech Dataset
Pipeline Scope	Dataset organization, validation, text/audio processing, alignment, feature extraction, dataset splitting, data loader preparation, and training batch setup
Main Control File	metadata/metadata.csv
Final Status	Prepared for future ASR model training
Document Type	Technical Documentation

By: Elaf A. Saeed

Executive Summary

This document provides a complete technical report for the Iraqi Speech Dataset processing pipeline. The pipeline converts raw audio and transcript files into a structured and training-ready speech dataset. It consists of ten stages, each stage performing a specific function and producing outputs that are consumed by later stages.

The central design principle of the pipeline is modularity. Each stage is implemented as a separate processing module, while the central metadata file stores the evolving state of each sample. This design makes the pipeline easier to debug, extend, document, and reproduce.



Document Structure

1. Project overview and pipeline architecture.
2. Detailed description of Stage 1 to Stage 10.
3. Inputs, processing logic, and outputs for every step.
4. Final pipeline outputs.
5. Future development roadmap.
6. Recommended next stages after Stage 10.

Pipeline at a Glance

Stage	Name	Main Objective
Stage 1	Data Organization	Organize raw audio and transcript files into a clean, standardized, paired, and traceable dataset structure.
Stage 2	Data Validation	Validate audio files, transcript files, and audio-transcript pairs before deeper processing.
Stage 3	Text Processing and Normalization	Clean, normalize, and prepare transcript text for speech processing and ASR training.
Stage 4	Audio Processing	Standardize and prepare audio files for alignment, feature extraction, and model

By: Elaf A. Saeed

		training.
Stage 5	Audio Preparation and Segmentation Decision	Analyze processed audio and decide whether segmentation is needed.
Stage 6	Forced Alignment	Align processed audio with transcripts at the word level.
Stage 7	Feature Extraction	Extract numerical acoustic features from processed audio files.
Stage 8	Dataset Preparation for Model Training	Organize extracted features and transcripts into train, validation, and test manifests.
Stage 9	Data Loader and Training Interface Preparation	Prepare labels, dataset objects, and reports needed before batch preparation.
Stage 10	Training Setup and Batch Preparation	Prepare training-ready datasets, batch padding, data loaders, and validation reports.

Central Metadata Design

The main metadata file is the backbone of the pipeline. It is progressively updated from Stage 1 to Stage 10. Instead of losing information between stages, each stage appends new columns or updates existing columns to describe the current status of each sample.

- Stage 1 creates the initial metadata file.
- Stage 2 adds validation results.
- Stage 3 adds processed transcript information.
- Stage 4 adds processed audio information.
- Stage 5 adds segmentation decision information.
- Stage 6 adds alignment information.
- Stage 7 adds feature extraction information.
- Stage 8 adds dataset split information.
- Stages 9 and 10 use the prepared manifests and vocabulary to build training interfaces and loaders.

By: Elaf A. Saeed

Stage 1 - Data Organization

Objective	Organize raw audio and transcript files into a clean, standardized, paired, and traceable dataset structure.
Main Inputs	Raw audio files Raw transcript files Project root directory
Main Outputs	metadata/file_inventory.csv metadata/rename_mapping.csv metadata/pair_index.csv metadata/metadata.csv
Workflow	Raw Files -> File Discovery -> Inventory -> Rename Mapping -> Rename Execution -> Pair Index -> Initial Metadata

Detailed Steps

Step 1 - Project Structure Creation

Objective: Create the required folders that will hold data, metadata, reports, logs, and source modules.

Processing:

- Check whether required folders exist.
- Create missing folders automatically.
- Keep the project structure consistent for all later stages.

Output:

- Standardized project directory structure.

Step 2 - File Discovery

Objective: Detect all available audio and transcript files inside the dataset directory.

Processing:

- Scan the selected dataset folder.
- Identify audio extensions such as .wav, .mp3, .flac, and .m4a.
- Identify transcript files such as .txt.

Output:

- Detected audio file list.
- Detected transcript file list.

Step 3 - File Inventory Generation

Objective: Build a complete inventory of discovered files.

Processing:

- Record file name, path, type, extension, and size.
- Separate audio and transcript file information clearly.

Output:

By: Elaf A. Saeed

- metadata/file_inventory.csv

Step 4 - Rename Planning

Objective: Prepare a consistent naming convention for all files.

Processing:

- Generate standardized IDs such as IQ_000001.
- Map original names to clean dataset names.

Output:

- metadata/rename_mapping.csv

Step 5 - Rename Execution

Objective: Apply the planned renaming operation.

Processing:

- Rename files according to the mapping table.
- Preserve traceability between original and new filenames.

Output:

- Standardized audio and transcript filenames.

Step 6 - Audio-Transcript Pair Generation

Objective: Pair each audio file with its matching transcript file.

Processing:

- Match files by standardized ID.
- Detect missing pairs if any.

Output:

- metadata/pair_index.csv

Step 7 - Metadata Creation

Objective: Create the first central metadata file.

Processing:

- Combine pair information, file information, and basic speaker/dataset attributes.
- Initialize columns required by later stages.

Output:

- metadata/metadata.csv

Step 8 - Stage Completion Check

Objective: Confirm that the dataset is organized and ready for validation.

By: Elaf A. Saeed

Processing:

- Check main output files.
- Log Stage 1 completion.

Output:

- Organized dataset ready for Stage 2.

Stage Output Summary

- metadata/file_inventory.csv
- metadata/rename_mapping.csv
- metadata/pair_index.csv
- metadata/metadata.csv

Future Improvements for This Stage

- Add duplicate audio fingerprint detection.
- Add automatic speaker ID extraction from folder names.
- Add dataset versioning for multiple collection rounds.

By: Elaf A. Saeed

Stage 2 - Data Validation

Objective	Validate audio files, transcript files, and audio-transcript pairs before deeper processing.
Main Inputs	metadata.csv Audio files Transcript files pair_index.csv
Main Outputs	Updated metadata.csv reports/validation_report.csv
Workflow	Metadata -> Audio Validation -> Transcript Validation -> Pair Validation -> Metadata Update -> Validation Report

Detailed Steps

Step 1 - Audio Validation

Objective: Check whether each audio file is readable and technically valid.

Processing:

- Verify file existence.
- Read audio metadata.
- Calculate duration, sample rate, channels, bit depth, RMS, peak, and silence ratio.

Output:

- audio_valid
- duration_sec
- sample_rate
- channels
- bit_depth
- rms
- peak
- silence_ratio

Step 2 - Transcript Validation

Objective: Check whether each transcript is readable and usable.

Processing:

- Verify file existence and readability.
- Check UTF-8 compatibility.
- Check whether text is empty or duplicated.
- Count characters, words, and lines.

Output:

- exists
- readable
- utf8
- empty
- duplicated

By: Elaf A. Saeed

- characters
- words
- lines
- transcript_valid

Step 3 - Pair Validation

Objective: Verify that every valid audio file has a matching transcript.

Processing:

- Compare audio_file and transcript_file pairs.
- Detect missing or broken pairs.

Output:

- pair_valid
- pair validation message

Step 4 - Metadata Update

Objective: Write validation results into the central metadata file.

Processing:

- Merge audio validation, transcript validation, and pair validation results.
- Preserve previous metadata columns.

Output:

- Updated metadata.csv

Step 5 - Validation Report Generation

Objective: Create a validation summary report.

Processing:

- Summarize total files, valid files, invalid files, warnings, and issues.

Output:

- reports/validation_report.csv

Stage Output Summary

- Updated metadata.csv
- reports/validation_report.csv

Future Improvements for This Stage

- Add language detection for transcript files.
- Add automatic corrupted audio repair suggestions.
- Add more detailed audio quality scoring.

By: Elaf A. Saeed

Stage 3 - Text Processing and Normalization

Objective	Clean, normalize, and prepare transcript text for speech processing and ASR training.
Main Inputs	Transcript files metadata.csv
Main Outputs	Processed transcript files Updated metadata.csv
Workflow	Raw Transcript -> Cleaning -> Normalization -> Iraqi Dialect Processing -> Quality Validation -> Save -> Metadata Update

Detailed Steps

Step 1 - Load Transcripts

Objective: Read transcript files from disk.

Processing:

- Open each transcript file.
- Load file name, file path, and text content.

Output:

- Transcript record list.

Step 2 - Text Cleaning

Objective: Remove text noise and formatting problems.

Processing:

- Remove extra spaces.
- Remove unwanted symbols or noisy formatting.
- Standardize line breaks.

Output:

- original_text
- clean_text
- changed

Step 3 - Text Normalization

Objective: Standardize text representation.

Processing:

- Normalize Arabic letters where needed.
- Standardize spacing and punctuation.
- Prepare consistent text for later alignment and training.

Output:

- normalized_text

By: Elaf A. Saeed

Step 4 - Iraqi Dialect Processing

Objective: Apply processing suitable for Iraqi dialect data.

Processing:

- Apply dialect-aware substitutions or text handling.
- Preserve useful spoken-language expressions.

Output:

- `processed_text`

Step 5 - Transcript Quality Validation

Objective: Check whether processed text is acceptable.

Processing:

- Reject empty text.
- Check word count and character count.
- Flag warnings for unusual transcripts.

Output:

- `status`
- `message`
- `word_count`
- `character_count`

Step 6 - Save Processed Transcripts

Objective: Save processed transcript outputs.

Processing:

- Write processed transcript files into the project outputs.

Output:

- Processed transcript files.

Step 7 - Update Metadata

Objective: Store processed text in metadata.

Processing:

- Update `transcript` and `normalized_text` columns.
- Set `text_processed` and `text_status`.

Output:

- `transcript`
- `normalized_text`
- `text_processed`

By: Elaf A. Saeed

- text_status

Stage Output Summary

- Processed transcript files
- Updated metadata.csv

Future Improvements for This Stage

- Add stronger Arabic normalization options.
- Add optional diacritics removal.
- Add text cleaning rules specific to Iraqi dialect variants.
- Add profanity/noise tagging if required by dataset policy.

By: Elaf A. Saeed

Stage 4 - Audio Processing

Objective	Standardize and prepare audio files for alignment, feature extraction, and model training.
Main Inputs	Audio files metadata.csv
Main Outputs	data/processed_audio/ Updated metadata.csv Audio processing report
Workflow	Raw Audio -> Load -> Standardize -> Enhance -> Silence Processing -> Quality Validation -> Save -> Metadata Update -> Report

Detailed Steps

Step 1 - Load Audio

Objective: Load valid audio files.

Processing:

- Read each audio file into memory.
- Prepare the signal for processing.

Output:

- Loaded audio records.

Step 2 - Audio Standardization

Objective: Convert audio into a consistent format.

Processing:

- Set target sample rate, usually 16000 Hz.
- Convert to mono.
- Use WAV-compatible processing.

Output:

- Standardized audio signal.

Step 3 - Audio Enhancement

Objective: Improve basic audio quality.

Processing:

- Normalize amplitude.
- Apply safe signal adjustment if needed.
- Preserve speech content.

Output:

- Enhanced audio signal.

Step 4 - Silence Processing

Objective: Analyze silence characteristics.

By: Elaf A. Saeed

Processing:

- Detect silence regions.
- Estimate silence ratio.
- Prepare statistics for later quality control.

Output:

- Silence statistics.

Step 5 - Audio Quality Validation

Objective: Validate processed audio quality.

Processing:

- Check duration, sample rate, signal validity, silence ratio, issues, and warnings.

Output:

- processing_status
- validation_status
- issues
- warnings

Step 6 - Save Processed Audio

Objective: Save standardized audio files.

Processing:

- Write processed audio into data/processed_audio/.

Output:

- Processed WAV files.

Step 7 - Update Metadata

Objective: Store processed audio information.

Processing:

- Update audio_processed, processed_sample_rate, processed_duration, and validation columns.

Output:

- Updated metadata.csv

Step 8 - Generate Processing Report

Objective: Generate audio processing report.

Processing:

- Summarize processed, failed, and warning samples.

By: Elaf A. Saeed

Output:

- Audio processing report.

Stage Output Summary

- data/processed_audio/
- Updated metadata.csv
- Audio processing report

Future Improvements for This Stage

- Add noise reduction as an optional module.
- Add LUFS loudness normalization.
- Add speaker diarization support for long recordings.

By: Elaf A. Saeed

Stage 5 - Audio Preparation and Segmentation Decision

Objective	Analyze processed audio and decide whether segmentation is needed.
Main Inputs	data/processed_audio/ metadata.csv
Main Outputs	Updated metadata.csv Segmentation report
Workflow	Processed Audio -> Dataset Analysis -> Type Detection -> Segmentation Decision -> Placeholder Segmentation -> Validation -> Metadata Update -> Report

Detailed Steps

Step 1 - Load Processed Audio

Objective: Load audio files produced by Stage 4.

Processing:

- Read processed audio paths and basic metadata.

Output:

- Processed audio record list.

Step 2 - Dataset Analysis

Objective: Analyze audio characteristics.

Processing:

- Calculate duration statistics.
- Inspect length distribution and sample rate consistency.

Output:

- Dataset analysis summary.

Step 3 - Dataset Type Detection

Objective: Detect whether the dataset is made of short utterances or long recordings.

Processing:

- Use duration and file distribution information.
- Classify dataset type.

Output:

- Dataset type decision.

Step 4 - Segmentation Decision

Objective: Decide whether segmentation is needed.

Processing:

- Use audio duration and dataset type.

By: Elaf A. Saeed

- Mark samples as no segmentation needed, recommended, or required.

Output:

- Segmentation decision.

Step 5 - Placeholder Segmentation

Objective: Simulate segmentation during development.

Processing:

- Estimate possible segment counts.
- Do not cut audio files in the current placeholder version.

Output:

- Placeholder segmentation output.

Step 6 - Segment Validation

Objective: Validate segmentation outputs or decisions.

Processing:

- Check segment count and segmentation status.

Output:

- segmented
- segment_count
- segmentation_status

Step 7 - Update Metadata

Objective: Store segmentation information.

Processing:

- Update segmented, segment_count, and related columns.

Output:

- Updated metadata.csv

Step 8 - Generate Segmentation Report

Objective: Generate a segmentation decision report.

Processing:

- Summarize dataset type and segmentation results.

Output:

- Segmentation report.

By: Elaf A. Saeed

Stage Output Summary

- Updated metadata.csv
- Segmentation report

Future Improvements for This Stage

- Replace placeholder segmentation with Silero VAD.
- Integrate WhisperX or Pyannote for long audio segmentation.
- Export real segment-level metadata.

By: Elaf A. Saeed

Stage 6 - Forced Alignment

Objective	Align processed audio with transcripts at the word level.
Main Inputs	data/processed_audio/ metadata.csv normalized_text
Main Outputs	data/alignment/ Updated metadata.csv reports/alignment_report.csv
Workflow	Processed Audio + Text -> Load -> Prepare -> Placeholder Alignment -> Validate -> Export -> Metadata Update -> Report

Detailed Steps

Step 1 - Load Audio and Transcript

Objective: Load processed audio with its corresponding text.

Processing:

- Read audio_file, audio path, normalized_text, duration, and sample rate.

Output:

- Alignment input dataset.

Step 2 - Alignment Preparation

Objective: Check if each sample is ready for alignment.

Processing:

- Verify audio exists.
- Verify transcript exists and is not empty.
- Check duration and sample rate.

Output:

- alignment_ready
- preparation_status

Step 3 - Forced Alignment

Objective: Generate word-level timestamps.

Processing:

- Split transcript into words.
- Use placeholder algorithm to distribute word times across duration.

Output:

- Word-level alignment records.

Step 4 - Alignment Validation

Objective: Validate generated alignment.

By: Elaf A. Saeed

Processing:

- Check word list, timestamp order, start and end values, and completion status.

Output:

- alignment_validation
- alignment_score

Step 5 - Export Alignment Files

Objective: Save alignment results.

Processing:

- Write JSON files under data/alignment/.

Output:

- Alignment JSON files.

Step 6 - Update Metadata

Objective: Store alignment status.

Processing:

- Update alignment_ready, preparation_status, alignment_status, alignment_score, and alignment_validation.

Output:

- Updated metadata.csv

Step 7 - Generate Alignment Report

Objective: Create alignment report.

Processing:

- Summarize completed, skipped, failed, and passed alignments.

Output:

- reports/alignment_report.csv

Stage Output Summary

- data/alignment/
- Updated metadata.csv
- reports/alignment_report.csv

Future Improvements for This Stage

- Replace placeholder alignment with WhisperX.
- Export TextGrid and CTM formats.
- Add phoneme-level alignment for advanced ASR research.

By: Elaf A. Saeed

By: Elaf A. Saeed

Stage 7 - Feature Extraction

Objective	Extract numerical acoustic features from processed audio files.
Main Inputs	data/processed_audio/ data/alignment/ metadata.csv
Main Outputs	data/features/ Updated metadata.csv reports/feature_report.csv
Workflow	Valid Audio -> Feature Config -> Extract Features -> Validate -> Save .npz -> Metadata Update -> Report

Detailed Steps

Step 1 - Load Valid Dataset

Objective: Load samples ready for feature extraction.

Processing:

- Select samples where audio_processed is True and alignment_validation is Passed.

Output:

- Valid feature extraction dataset.

Step 2 - Feature Preparation

Objective: Prepare feature extraction configuration.

Processing:

- Set feature_type, sample_rate, output folder, n_mfcc, n_mels, n_fft, hop_length, and file format.

Output:

- Feature configuration dictionary.

Step 3 - Audio Feature Extraction

Objective: Extract selected acoustic features.

Processing:

- MFCC mode extracts MFCC, Delta, and Delta-Delta.
- Mel mode extracts Log-Mel Spectrogram.
- Combined mode extracts MFCC-related features, RMS, ZCR, and Spectral Centroid.

Output:

- Feature arrays.

Step 4 - Feature Validation

Objective: Validate feature arrays before saving.

Processing:

By: Elaf A. Saeed

- Check not None, not empty, 2D shape, no NaN, no Inf, and valid frames.

Output:

- feature_valid
- validation_status
- validation_message

Step 5 - Save Features

Objective: Save features as .npy files.

Processing:

- Save valid features under data/features/<feature_type>/.

Output:

- Feature .npy files.

Step 6 - Update Metadata

Objective: Store feature information.

Processing:

- Update feature_extracted, feature_type, feature_file, feature_shape, feature_status, and feature_message.

Output:

- Updated metadata.csv

Step 7 - Generate Feature Report

Objective: Generate feature extraction report.

Processing:

- Summarize saved, skipped, failed, and validated features.

Output:

- reports/feature_report.csv

Stage Output Summary

- data/features/
- Updated metadata.csv
- reports/feature_report.csv

Future Improvements for This Stage

- Add Whisper embeddings.
- Add Wav2Vec2 or HuBERT feature extraction.
- Add feature normalization and CMVN.
- Add optional spectrogram image export for visualization.

By: Elaf A. Saeed

By: Elaf A. Saeed

Stage 8 - Dataset Preparation for Model Training

Objective	Organize extracted features and transcripts into train, validation, and test manifests.
Main Inputs	metadata.csv Feature files Processed transcripts
Main Outputs	data/dataset_splits/train_manifest.csv data/dataset_splits/val_manifest.csv data/dataset_splits/test_manifest.csv Updated metadata.csv reports/dataset_preparation_report.csv
Workflow	Feature Metadata -> Training Records -> Split -> Save Manifests -> Validate -> Metadata Update -> Report

Detailed Steps

Step 1 - Load Feature Dataset

Objective: Load samples with saved features and usable text.

Processing:

- Select samples where feature_extracted is True, feature_status is Saved, feature_file exists, and text exists.

Output:

- feature_dataset.

Step 2 - Prepare Training Records

Objective: Convert samples into structured training records.

Processing:

- Prepare training_text, word_count, text_length, duration, sample_rate, and record status.

Output:

- training_records.

Step 3 - Split Dataset

Objective: Split data into train, validation, and test subsets.

Processing:

- Shuffle with random_state=42.
- Use 70 percent train, 15 percent validation, 15 percent test.

Output:

- train_records
- val_records
- test_records

Step 4 - Save Dataset Manifests

Objective: Save split records as CSV manifests.

By: Elaf A. Saeed

Processing:

- Write train_manifest.csv, val_manifest.csv, and test_manifest.csv.

Output:

- data/dataset_splits/*.csv

Step 5 - Validate Dataset Splits

Objective: Validate manifest files.

Processing:

- Check files exist, are not empty, feature files exist, training_text is not empty, split labels are correct, and no overlap exists.

Output:

- validation_summary.

Step 6 - Update Metadata

Objective: Store split information in metadata.

Processing:

- Update dataset_split, included_in_training, dataset_status, and dataset_message.

Output:

- Updated metadata.csv

Step 7 - Generate Dataset Report

Objective: Create dataset preparation report.

Processing:

- Summarize split counts, ratios, average words, duration, manifest paths, and validation status.

Output:

- reports/dataset_preparation_report.csv

Stage Output Summary

- data/dataset_splits/train_manifest.csv
- data/dataset_splits/val_manifest.csv
- data/dataset_splits/test_manifest.csv
- Updated metadata.csv
- reports/dataset_preparation_report.csv

Future Improvements for This Stage

- Add stratified split by speaker, gender, dialect, or governorate.

By: Elaf A. Saeed

- Add speaker-independent split mode.
- Add k-fold cross-validation support.

By: Elaf A. Saeed

Stage 9 - Data Loader and Training Interface Preparation

Objective	Prepare labels, dataset objects, and reports needed before batch preparation.
Main Inputs	train_manifest.csv val_manifest.csv test_manifest.csv Feature files training_text
Main Outputs	data/training_interface/character_vocabulary.json reports/data_loader_report.csv reports/data_loader_records_report.csv
Workflow	Manifests -> Load Features -> Character Labels -> Dataset Objects -> Validate -> Reports

Detailed Steps

Step 1 - Load Dataset Manifests

Objective: Load train, validation, and test manifests.

Processing:

- Verify files exist, required columns exist, and split labels are correct.

Output:

- train_df
- val_df
- test_df
- manifest_summary

Step 2 - Load Feature Files

Objective: Load .npy feature files from manifest paths.

Processing:

- Resolve paths, load NumPy arrays, check not empty.

Output:

- train_features
- val_features
- test_features

Step 3 - Prepare Text Labels

Objective: Convert training text into numerical character-level labels.

Processing:

- Build character vocabulary.
- Assign <blank>=0 and <unk>=1.
- Encode each transcript as label IDs.

Output:

By: Elaf A. Saeed

- character_vocabulary.json
- label_ids
- label_length

Step 4 - Build Dataset Objects

Objective: Build structured dataset objects.

Processing:

- Combine features, label_ids, label_text, feature_shape, feature_dim, feature_length, and split.

Output:

- train_dataset objects
- val_dataset objects
- test_dataset objects

Step 5 - Validate Data Loader

Objective: Validate dataset objects before batch preparation.

Processing:

- Check features, labels, lengths, NaN/Inf values, and CTC compatibility.

Output:

- data_loader_validation_status.

Step 6 - Generate Data Loader Report

Objective: Generate summary and record-level reports.

Processing:

- Write data_loader_report.csv and data_loader_records_report.csv.

Output:

- Reports under reports/.

Stage Output Summary

- data/training_interface/character_vocabulary.json
- reports/data_loader_report.csv
- reports/data_loader_records_report.csv

Future Improvements for This Stage

- Add word-piece or BPE tokenization.
- Add language model vocabulary preparation.
- Add saved serialized dataset indexes for faster loading.

By: Elaf A. Saeed

Stage 10 - Training Setup and Batch Preparation

Objective	Prepare training-ready datasets, batch padding, data loaders, and validation reports.
Main Inputs	train_manifest.csv val_manifest.csv test_manifest.csv character_vocabulary.json Feature files
Main Outputs	train_loader val_loader test_loader reports/training_setup_report.csv
Workflow	Resources -> ASRDataset -> Collate Function -> DataLoaders -> Batch Validation -> Training Setup Report

Detailed Steps

Step 1 - Load Training Resources

Objective: Load resources needed for training setup.

Processing:

- Load train/val/test manifests.
- Load character_vocabulary.json.
- Validate manifests and vocabulary.

Output:

- training_resources
- resource_summary

Step 2 - Build ASR Dataset

Objective: Build ASR dataset objects.

Processing:

- Create train_dataset, val_dataset, and test_dataset.
- Load feature file path and encode labels.
- Transpose features from (feature_dim, time_frames) to (time_frames, feature_dim).

Output:

- train_dataset
- val_dataset
- test_dataset

Step 3 - Create Batch Collate Function

Objective: Create padding function for variable-length samples.

Processing:

- Pad features to max time length.
- Pad labels to max label length.

By: Elaf A. Saeed

- Return input_lengths, label_lengths, flat_labels, and metadata.

Output:

- asr_collate_fn and tested collated batches.

Step 4 - Build DataLoaders

Objective: Build train, validation, and test loaders.

Processing:

- Use PyTorch DataLoader if available; otherwise use SimpleDataLoader.
- Set train shuffle=True and val/test shuffle=False.
- Use batch_size=4 and num_workers=0.

Output:

- train_loader
- val_loader
- test_loader

Step 5 - Validate Training Batches

Objective: Validate batches produced by loaders.

Processing:

- Check features are 3D, labels are 2D, flat labels are 1D, lengths are valid, no NaN/Inf, and input_length >= label_length.

Output:

- batch_validation_summary

Step 6 - Generate Training Setup Report

Objective: Generate final training readiness report.

Processing:

- Summarize resources, datasets, collate, loaders, batch validation, and final readiness.

Output:

- reports/training_setup_report.csv

Stage Output Summary

- train_loader
- val_loader
- test_loader
- reports/training_setup_report.csv

By: Elaf A. Saeed

Future Improvements for This Stage

- Add GPU-ready tensor conversion.
- Add configurable batch size and num_workers from config file.
- Add bucketing by audio length for faster training.
- Add Stage 11 ASR model training.

By: Elaf A. Saeed

Final Outputs of the Complete Pipeline

After completing Stage 1 to Stage 10, the project produces the following main outputs:

- metadata/metadata.csv
- data/processed_audio/
- data/alignment/
- data/features/
- data/dataset_splits/train_manifest.csv
- data/dataset_splits/val_manifest.csv
- data/dataset_splits/test_manifest.csv
- data/training_interface/character_vocabulary.json
- reports/validation_report.csv
- reports/alignment_report.csv
- reports/feature_report.csv
- reports/dataset_preparation_report.csv
- reports/data_loader_report.csv
- reports/data_loader_records_report.csv
- reports/training_setup_report.csv

Final Technical Status

At the end of Stage 10, the dataset is no longer only a collection of raw audio and text files. It becomes a structured ASR-ready dataset with validated audio, processed text, alignment information, extracted features, dataset splits, character-level labels, dataset objects, padded batches, and data loaders.

Component	Status After Stage 10
Audio files	Processed and standardized
Transcript files	Cleaned, normalized, and linked to metadata
Alignment	Available using placeholder alignment, ready to be replaced with WhisperX
Features	Extracted and saved as .npy files
Dataset split	Train / validation / test manifests created
Labels	Character-level labels and vocabulary prepared
Data loaders	Built and validated
Training readiness	Ready for future ASR model training

By: Elaf A. Saeed

Future Development Roadmap

The current pipeline provides a complete data preparation foundation. The following future improvements are recommended to transform it from a development-ready pipeline into a production-level ASR research platform.

Real Forced Alignment Integration

- Replace the current placeholder alignment with WhisperX.
- Add TextGrid and CTM export formats.
- Add confidence scores for each aligned word.

Real Audio Segmentation

- Replace placeholder segmentation with Silero VAD or Pyannote.
- Support long recordings and automatic utterance extraction.
- Add segment-level metadata and reports.

Advanced Feature Extraction

- Add Whisper, Wav2Vec2, and HuBERT embeddings.
- Add CMVN and feature normalization.
- Support multiple feature versions in the same project.

Dataset Quality Dashboard

- Build a dashboard to visualize duration distribution, transcript length, validation errors, and split balance.
- Add charts for speaker, gender, dialect, and governorate distribution.

Speaker and Dialect Balancing

- Add speaker-independent train/val/test splitting.
- Balance data across gender, dialect, and governorate.
- Detect speaker leakage between splits.

ASR Model Training

- Create Stage 11 for ASR model training.
- Start with CTC-based BiLSTM or Transformer model.
- Track WER, CER, training loss, and validation loss.

Model Evaluation and Error Analysis

- Create Stage 12 for evaluation.
- Generate WER/CER reports.
- Produce word-level and character-level error analysis.

Experiment Tracking and MLOps

- Add MLflow for experiment tracking.
- Store model versions and training parameters.
- Add reproducible configuration files.

Data Augmentation

- Add speed perturbation, noise injection, and volume perturbation.

By: Elaf A. Saeed

- Create augmentation metadata to track original and augmented samples.

Deployment Preparation

- Export trained model to ONNX or TorchScript.
- Prepare FastAPI inference endpoint.
- Create simple web interface for Iraqi speech transcription.

Security and Privacy

- Add speaker anonymization if required.
- Add dataset license and consent tracking.
- Add sensitive text filtering policy.

Documentation and Reproducibility

- Generate automatic documentation from pipeline logs.
- Add README files for each stage.
- Add a master configuration file for all paths and parameters.

Recommended Next Stages

Proposed Stage	Purpose	Main Output
Stage 11 - ASR Model Training	Train the first speech recognition model using the prepared loaders.	Trained ASR model and training history
Stage 12 - Model Evaluation	Evaluate ASR performance using WER and CER.	Evaluation report and error analysis
Stage 13 - Model Optimization	Improve accuracy and inference speed.	Optimized model
Stage 14 - Deployment Interface	Deploy the model through API or simple application.	Inference service or demo interface

Conclusion

The Iraqi Speech Dataset pipeline successfully establishes a full end-to-end preparation system for ASR research. It begins with raw files and ends with training-ready batches and data loaders. Each stage is modular, traceable, and documented. The project is now ready to move toward actual ASR model training and evaluation.